

School Management App

API Resource Réteg – Fejlesztői dokumentáció

Laravel 12 + Clean Code elvek | 2026

1. Mi az API Resource és miért kell?

Az API Resource a Model és a JSON válasz között helyezkedik el. Feladata hogy pontosan meghatározza mit és hogyan adunk vissza a frontendnek – nem az összes adatbázis mezőt, hanem csak a szükségeseket, formázva.

1.1 A probléma – Model visszaadása közvetlenül

```
// ROSSZ – közvetlen Model visszaadás
return Student::find(1);

// Ezt kapja vissza a frontend:
{
  "id": 1,
  "name": "Kovács Anna",
  "email": "anna@example.com",
  "date_of_birth": "2010-05-12T00:00:00.000000Z",
  "class_id": 3,
  "created_at": "2026-06-05T10:00:00.000000Z",
  "updated_at": "2026-06-05T10:00:00.000000Z",
  "deleted_at": null
}
```

Problémák:

- created_at, updated_at, deleted_at → belső mezők, frontend nem kell
- class_id → csak szám, az osztály neve kellene
- Nincs kontroll → ha hozzáadunk új mezőt, az azonnal megjelenik
- Beágyazott adatok → ha load-oljuk a schoolClass-t, az egész objektum bekerül

1.2 A megoldás – API Resource

```
// JÓ – Resource-on keresztül
```

```

return new StudentResource(Student::find(1));

// Ezt kapja vissza a frontend:
{
  "data": {
    "id": 1,
    "name": "Kovács Anna",
    "email": "anna@example.com",
    "roll_number": "STU-2026-00001",
    "school_class": "10A",
    "created_at": "2026-06-05"
  }
}

```

2. A \$this titka – Magic Method

A Resource-ban a \$this nem a Resource osztály saját mezőjét jelenti – hanem a hozzá tartozó Model mezőjét! Ez a JsonResponse őssosztály egy __get() magic method-ja segítségével működik.

```

class StudentResource extends JsonResponse
{
    public function toArray(Request $request): array
    {
        return [
            'name' => $this->name,
            // ↓ valójában ez:
            // 'name' => $this->resource->name
            // ↓ vagy rövidítve: $this->name
        ];
    }
}

```

Miért működik a magic method így?

A JsonResponse alaposztály __get() metódusa azt csinálja: ha a Resource-on egy ismeretlen property-re hivatkozunk (pl. \$this->name), automatikusan az \$this->resource objektumra átirányítja (\$this->resource->name). Ez azért hasznos, mert nem kell mindig \$this->resource->name-et írni – elég \$this->name-et. Ez a "magic" – a háttérben valami más történik, de az szintaxis elegáns marad.

3. whenLoaded() – Az N+1 probléma védelme

Az összes Resource-on végig használjuk a whenLoaded() metódust a kapcsolatoknál. Ez a legfontosabb dolog amit meg kell érteni – a Resource réteg N+1 védelmének utolsó vonala.

3.1 A probléma – Resource N+1 nélkül

```
// Service - elfelejtettük a with(['schoolClass'])-t
public function getAllStudents(): LengthAwarePaginator
{
    return Student::paginate(15); // NINCS eager loading!
}

// Resource - közvetlenül elérjük a schoolClass-t
public function toArray(Request $request): array
{
    return [
        'school_class' => $this->schoolClass->name,
        // Minden diáknál EGY újabb lekérdezés!
        // 15 diák = 15 extra lekérdezés = N+1 probléma
    ];
}
```

3.2 A megoldás – whenLoaded()

```
// Resource - defendel magunkat a whenLoaded()-val
public function toArray(Request $request): array
{
    return [
        'school_class' => $this->whenLoaded('schoolClass', function () {
            return [
                'id' => $this->schoolClass->id,
                'name' => $this->schoolClass->name,
            ];
        }),
    ];
}

// Mi történik:
// - Ha BE VAN töltve eagerloading-gal → visszaadja az adatot (0 extra lekérdezés)
// - Ha NINCS töltve → kihagyja ezt a mezőt, NEM futtat új lekérdezést
```

Miért whenLoaded() az előnye?

A whenLoaded() a Service réteg felelőssége: gondoskodnia kell az eager loading-ról. Ha elfelejtette, a whenLoaded() nem panaszkodik és nem futtat újabb lekérdezéseket – csak kihagyja a mezőt. Így még Resource nélküli kódban sem okozhat N+1 problémát. Ez egy "fail-safe" megoldás – ha valami nem működik, nem fut le, de nem zuhan be az alkalmazás.

4. whenLoaded() vs when() – Mikor mit használunk?

Két hasonló metódus, de különböző céllal. A `whenLoaded()` kapcsolatokhoz, a `when()` feltételes mezőkhöz.

Metódus	Mire usar	Példa	Mi történik ha hamis
<code>whenLoaded('relation')</code>	Eloquent kapcsolatok ellenőrzése	<code>whenLoaded('schoolClass')</code>	Kihagyja a mezőt, nincs új lekérdezés
<code>when(condition)</code>	Bármilyen boolean feltétel	<code>when(isset(\$this->count))</code>	Kihagyja a mezőt, nincs adat
<code>whenLoaded()</code>	N+1 védelem egyúttal	Kapcsolatok	Biztonságos – nincs extra query
<code>when()</code>	Egyszerű logika	Számított mezők, flagi	Egyszerű boolean ellenőrzés

5. Carbon Cast-ok és a Resource – `isPast()`, `isFuture()`

A Model-ben definiált `$casts` miatt a Resource-ban Carbon objektumokkal dolgozunk. A Carbon rengeteg hasznos metódust nyújt a dátumok kezelésére.

5.1 Mik azok a Carbon objektumok?

```
// Model:
protected $casts = [
    'due_date' => 'date',          // ← Carbon objektum lesz
    'paid_date' => 'date',        // ← Carbon objektum lesz
];

// Resource-ban:
'due_date' => $this->due_date?->format('Y-m-d'),
// $this->due_date már Carbon objektum, a ?-> biztonságos hozzáférés null-ra
```

5.2 Hasznos Carbon metódusok a Resource-ban

Metódus	Mit csinál	Visszatérés	Projektben
<code>->format('Y-m-d')</code>	Formázza a dátumot	string	Minden dátum mező
<code>->format('H:i')</code>	Formázza az időt	string	<code>start_time</code> , <code>end_time</code>
<code>->isPast()</code>	Lejárt-e már?	boolean	<code>FeeResource</code> , <code>LibraryIssueResource</code>

->isFuture()	Jövőbeli-e?	boolean	Hasonlítható isPast()-hez
->isToday()	Ma van-e?	boolean	Napok szerinti szűréshez
->diffInDays(now())	Napi különbség	integer	Lejárat napok számítása
->age	Életkor (geboren-hoz)	integer	Diákok életkora

isPast() a FeeResource-ban

'is_overdue' => \$this->status === 'pending' && \$this->due_date->isPast(), – Ez a számított mező a Resource-ban azt ellenőrzi: ha a díj még pending ÉS a határidő már lejárt, akkor overdue. A Frontend így nem kell hogy saját maga számítsa ki – csak mutatja meg az "Lejárt" flagi-t.

6. Nullsafe Operator – ?->

A nullsafe operator (?->) PHP 8.0-ban jelent meg. Biztonságosan meghívja a metódusokat még akkor is, ha az objektum null.

```
// PHP 7 - nem nullsafe:
isset($this->date_of_birth) ? $this->date_of_birth->format('Y-m-d') : null

// PHP 8+ - nullsafe operator:
$this->date_of_birth?->format('Y-m-d')
// ↓
// Ha $this->date_of_birth null → null
// Ha van érték → meghívja a format() metódust
```

A projektben ezeket használjuk:

```
'date_of_birth' => $this->date_of_birth?->format('Y-m-d'),
'joining_date'  => $this->joining_date?->format('Y-m-d'),
'start_time'    => $this->start_time?->format('H:i'),
'return_date'   => $this->return_date?->format('Y-m-d'),
```

7. Számított (derived) mezők – when() és logika

A Resource-ban készíthetünk olyan mezőket, amelyek nincsenek az adatbázisban – a Resource-ban számítjuk ki őket az elérhető mezőkből.

7.1 is_available – LibraryBookResource

```

public function toArray(Request $request): array
{
    return [
        'total_copies'      => $this->total_copies,
        'available_copies' => $this->available_copies,

        // Ez a mező nem az adatbázisban van!
        'is_available' => $this->available_copies > 0,
        //                ↑ számított mező - boolean logika
    ];
}

```

7.2 is_overdue – FeeResource és LibraryIssueResource

```

// FeeResource
'is_overdue' => $this->status === 'pending' && $this->due_date->isPast(),

// LibraryIssueResource
'is_overdue' => $this->status === 'issued' && $this->due_date->isPast(),

```

Miért számított mezők a Resource-ban?

Mert ezek "helper" adatok a frontendnek – a frontend könnyebben tudja a UI-t formázni ha már ezek az információ JSON-ben jelen vannak. Helyett hogy a frontend saját maga nézi meg a due_date-et és az isPast()-et, a Resource már elvégzi ezt. DRY (Don't Repeat Yourself) elv – a logika egy helyen van.

8. Pénzösszegek formázása – number_format()

Az amount és fine mezőkre decimal cast van – de JSON-ban stringként adjuk vissza hogy elkerüljük a JavaScript lebegőpontos kerekítési problémáit.

```

// FeeResource
'amount' => number_format($this->amount, 2, '.', ''),
//                ↑       ↑       ↑
//                value decimal separator thousands

// Példák:
// 15000      → '15000.00'
// 15000.5    → '15000.50'
// 15000.123 → '15000.12'

```

Miért number_format() stringként?

A JSON Number típusban a JavaScript lebegőpontos számokkal dolgozik, ami kerekítési hibákat okozhat pénzügyi adatoknál. Ha stringként adjuk vissza, a frontend pontosan azt mutatja amit szeretnénk – nem szükséges konverzió. A számológép/DB műveletek Laravelben decimalként futnak le (ahol pontos), a frontendre pedig szöveggént jön az adat (ahol megjelenítésre van szükség).

9. Collection() vs New – Lista vs egyetlen elem

A Controller alapján különbözik hogyan adjuk vissza a Resource-okat.

Eset	Megoldás	Pl. Controller	JSON szerkezet
Egyetlen elem	<code>new StudentResource(\$student)</code>	<code>getStudentById()</code>	<code>{"data": {...}}</code>
Lista (paginate)	<code>StudentResource::collection(\$students)</code>	<code>getAllStudents()</code>	<code>{"data": [...], "meta": {...}, "links": {...}}</code>
Lista (all)	<code>StudentResource::collection(\$students)</code>	<code>getAllWithRelations()</code>	<code>{"data": [...]}</code>

```
// Egyetlen elem:
return new StudentResource($student);

// Lista lapozással:
return StudentResource::collection($students); // $students a paginate()
eredménye

// Automatikus lapozási info:
{
  "data": [{...}, {...}],
  "links": {"first": ".../students?page=1", "last":
".../students?page=7"},
  "meta": {"current_page": 1, "last_page": 7, "per_page": 15, "total":
100}
}
```

10. Beágyazott Resource-ok – amikor további adat kell

Néha egy kapcsolatnál több adat is kell a frontendnek – nem csak id és name, hanem más mezők is. Ilyenkor nem egy sima array-t adunk vissza, hanem beágyazott Resource-t.

10.1 Egy kapcsolat – csak néhány mező (inline array)

```
// SubjectResource - tanár neve elég, nincs további adat
'teacher' => $this->whenLoaded('teacher', function () {
    return [
        'id' => $this->teacher->id,
        'name' => $this->teacher->name,
    ];
}),
```

10.2 Sok kapcsolat – teljes Resource beágyazás

```
// StudentResource - SchoolClass összes adatára szükség van
'school_class' => $this->whenLoaded('schoolClass', function () {
    return new SchoolClassResource($this->schoolClass);
    // ↑ teljes Resource a formázást kezeli
}),

// GuardianResource-ok listája
'guardians' => $this->whenLoaded('guardians', function () {
    return GuardianResource::collection($this->guardians);
}),
```

11. Kerülendő hibák Resource-ban

Hiba	Mi történik	Megoldás
Nincs whenLoaded()	N+1 probléma ha egyenlőtlen eager loading	Mindig whenLoaded() a kapcsolatokhoz
Körkörös beágyazás	StudentResource → SchoolClassResource → StudentResource	Csak az egyik oldalon adjuk vissza a kapcsolatot
Pénz float-ként	Kerekítési hibák, pl. 9.99 helyett 9.989999...	number_format() stringként
Nincs nullsafe	Exception ha mező null	?-> operátor nullable mezőknél
Felesleges mezők	Túl nagy JSON válasz, lassú frontend	Csak szükséges mezőket adjuk vissza

12. Összefoglaló – Resource best practices

Elvezet	Alkalmazás
---------	------------

\$this = Model tulajdonságok	Szépen és biztonságosan hozzáférhető
whenLoaded() az összes kapcsolathoz	N+1 probléma megelőzése
when() feltételes mezőkhöz	Dinamikus JSON szerkezet
?-> nullable operator	Nullable mezőknél exception helyett null
number_format() pénz mezőknél	Pontosság, JavaScript biztonság
Számított mezők	Frontend könnyebb UI formázása
Collection() lista returnnál	Automatikus lapozási info
Beágyazott Resource-ok	Szép hierarchia a JSON-ban

– School Management App fejlesztési dokumentáció –